LAPORAN PRATIKUM STRUKTUR DATA

“ STACK ARRAY”

OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T
ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI INFORMASI DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data stack serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Penjelasan Kode Program 3
2.2.1 Class StackArray_2511531007 3
2.2.2 Class StackArrayDriver_2511531007 4
2.2.3 Class StackPostfix_2511531007 4
2.2.4 Class NilaiMaksimum_2511531007 5
2.2.5 Class SiswaStack_2511531007 5
2.3 Tugas Praktikum 6
2.3.1 Class Website_2511531007 6
2.3.2 Class Browser_2511531007 7
BAB III PENUTUP 9
3.1 Kesimpulan 9
3.2 Saran 9
DAFTAR PUSTAKA 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut kemampuan dalam mengelola data secara efisien. Salah satu konsep penting dalam ilmu komputer adalah struktur data, yaitu cara menyimpan dan mengatur data agar dapat diakses dan diproses dengan baik. Pemahaman terhadap struktur data menjadi dasar dalam pengembangan program yang efektif dan optimal.

Stack adalah struktur data linear dengan prinsip Last In First Out (LIFO), di mana elemen terakhir yang masuk akan keluar lebih dulu. Struktur ini banyak digunakan dalam berbagai aplikasi seperti manajemen memori, pemanggilan fungsi, dan evaluasi ekspresi postfix, sehingga penting untuk dipahami.

Dalam praktikum ini, stack diimplementasikan menggunakan array, kelas bawaan Java, dan ArrayList, serta diterapkan pada kasus seperti pencarian nilai maksimum dan evaluasi ekspresi. Kegiatan ini bertujuan agar mahasiswa dapat memahami dan mengimplementasikan konsep stack secara langsung.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dasar struktur data stack beserta prinsip kerjanya (LIFO) dalam pemrograman.
2. Mengimplementasikan operasi-operasi dasar stack seperti push, pop, dan peek menggunakan bahasa pemrograman Java.

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Memahami cara pengelolaan data secara terstruktur dan efisien menggunakan stack.
2. Meningkatkan kemampuan dalam mengembangkan program sederhana yang menerapkan konsep struktur data dalam penyelesaian masalah.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Stack merupakan salah satu struktur data linear yang memiliki prinsip kerja LIFO (*Last In First Out*)**,** yaitu elemen yang terakhir dimasukkan ke dalam struktur akan menjadi elemen pertama yang dikeluarkan. Konsep ini dapat dianalogikan seperti tumpukan buku, di mana buku yang terakhir diletakkan di atas akan menjadi buku pertama yang diambil.

Dalam implementasinya, stack memiliki beberapa operasi dasar, yaitu push**,** pop**,** dan peek. Operasi *push* digunakan untuk menambahkan elemen ke dalam stack, biasanya pada bagian paling atas. Operasi *pop* digunakan untuk menghapus elemen teratas dari stack, sedangkan *peek* digunakan untuk melihat elemen paling atas tanpa menghapusnya. Selain itu, terdapat operasi tambahan seperti *isEmpty* untuk memeriksa apakah stack dalam kondisi kosong.

Stack dapat diimplementasikan menggunakan berbagai struktur data, seperti array maupun linked list. Pada implementasi berbasis array, stack menggunakan indeks tertentu (biasanya disebut *top*) untuk menunjukkan posisi elemen terakhir. Ketika dilakukan operasi *push*, nilai *top* akan bertambah, sedangkan saat *pop*, nilai *top* akan berkurang. Namun, implementasi ini memiliki keterbatasan pada kapasitas yang bersifat tetap. Sementara itu, implementasi menggunakan struktur dinamis seperti ArrayList atau linked list memungkinkan ukuran stack bertambah secara fleksibel sesuai kebutuhan.

Dalam bahasa pemrograman Java, stack dapat diimplementasikan menggunakan kelas bawaan seperti Stack, maupun dengan pendekatan manual menggunakan struktur lain seperti ArrayList. Penggunaan stack banyak diterapkan dalam berbagai kasus, seperti evaluasi ekspresi matematika (misalnya postfix), manajemen pemanggilan fungsi (call stack), serta proses pembalikan data.

Dengan memahami konsep stack, pengembang dapat mengelola data secara efisien dalam situasi yang membutuhkan akses elemen secara terstruktur dan terurut berdasarkan urutan masuknya data.

2.2 Penjelasan Kode Program

2.2.1 Class StackArray_2511531007

```java
package pekan3_2511531007;

public class stackArray_2511531007 {
    static final int MAX_1007 = 1000;
    int top_1007;
    int a_1007 [] = new int [MAX_1007];
    boolean isEmpty_1007()
    {
        return (top_1007< 0);
    }
    stackArray_2511531007()
    {
        top_1007 = -1;
    }
    boolean push_1007(int x_1007)
    {
        if (top_1007 >= (MAX_1007 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a_1007 [++top_1007] = x_1007;
            System.out.println(x_1007 + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_1007()
    {
        if (top_1007 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x_1007 = a_1007[top_1007--];
            return x_1007;
        }
    }
    int peek_1007()
    {
        if (top_1007 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x_1007 = a_1007[top_1007];
            return x_1007;
        }
    }
    void print_1007() {
        for (int i_1007 = top_1007; i_1007 > -1; i_1007--) {
        System.out.print(" " + a_1007[i_1007]);
        }
    }
}
```

Class stackArray_2511531007 merupakan implementasi struktur data stack menggunakan array sebagai media penyimpanan. Pada class ini terdapat atribut berupa array a_1007 untuk menyimpan elemen dan variabel top_1007 yang berfungsi sebagai penunjuk posisi elemen teratas dalam stack. Class ini menyediakan beberapa method utama seperti push_1007() untuk menambahkan elemen ke dalam stack, pop_1007() untuk menghapus elemen teratas, peek_1007() untuk melihat elemen paling atas tanpa menghapusnya, serta isEmpty_1007() untuk mengecek apakah stack dalam kondisi kosong. Selain itu, terdapat juga method print_1007() yang digunakan untuk menampilkan seluruh isi stack.

2.2.2 Class StackArrayDriver_2511531007

```java
package pekan3_2511531007;

public class stackArrayDriver_2511531007 {

    public static void main(String[] args) {
        stackArray_2511531007 s_1007 = new stackArray_2511531007();
        s_1007.push_1007(10);
        s_1007.push_1007(20);
        s_1007.push_1007(30);
        System.out.println(s_1007.pop_1007() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah :" + s_1007.peek_1007());
        System.out.print("Elemen pada stack :");
        s_1007.print_1007();
    }

}
```

Class stackArrayDriver_2511531007 berfungsi sebagai class utama (driver) yang digunakan untuk menjalankan dan menguji implementasi stack berbasis array. Pada class ini terdapat method main yang berisi proses pembuatan objek dari class stackArray_2511531007, kemudian dilakukan beberapa operasi seperti menambahkan elemen (*push*), menghapus elemen (*pop*), melihat elemen teratas (*peek*), serta menampilkan isi stack. Class ini bertujuan untuk menunjukkan bagaimana cara penggunaan stack serta memastikan bahwa setiap method pada class stack berjalan dengan baik sesuai dengan konsep yang telah dirancang.

2.2.3 Class StackPostfix_2511531007

```java
package pekan3_2511531007;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511531007 {

    public static int postfixEvaluate_1007(String expression_1007) {
        Stack<Integer> s_1007 = new Stack<Integer>();
        Scanner input_1007 = new Scanner(expression_1007);
        while (input_1007.hasNext()) {
            if (input_1007.hasNextInt()) { // an operand (integer)
                s_1007.push(input_1007.nextInt());
            } else {
                String operator_1007 = input_1007.next();
                int operand2_1007 = s_1007.pop();
                int operand1_1007 = s_1007.pop();
                if (operator_1007.equals("+")) {
                    s_1007.push(operand1_1007 + operand2_1007);
                } else if (operator_1007.equals("-")) {
                    s_1007.push(operand1_1007 - operand2_1007);
                } else if (operator_1007.equals("*")) {
                    s_1007.push(operand1_1007 * operand2_1007);
                } else {
                    s_1007.push(operand1_1007 / operand2_1007);
                }
            }
        }
        input_1007.close();
        return s_1007.pop();

    }
    public static void main(String[] args) {
        System.out.println("hasil postfix = "+postfixEvaluate_1007("5 2 4 * + 7 -"));
    }

}
```

Class StackPostfix_2511531007 digunakan untuk mengevaluasi ekspresi matematika dalam bentuk postfix menggunakan struktur data stack. Dalam class ini, input ekspresi diproses menggunakan `Scanner` dengan membaca setiap token secara berurutan. Jika token berupa angka, maka akan dimasukkan ke dalam stack. Namun jika token berupa operator, maka dua elemen teratas akan diambil dari stack untuk

dilakukan operasi sesuai operator tersebut, kemudian hasilnya dimasukkan kembali ke dalam stack. Proses ini dilakukan hingga seluruh ekspresi selesai diproses, dan hasil akhir diperoleh dari elemen terakhir dalam stack.

2.2.4 Class NilaiMaksimum_2511531007

```java
package pekan3_2511531007;

import java.util.Stack;

public class NilaiMaksimum_2511531007 {
    public static int max_1007(Stack<Integer> s_1007) {
        Stack<Integer> backup_1007 = new Stack<Integer>();
        int maxValue_1007 = s_1007.pop();
        backup_1007.push(maxValue_1007);
        while (!s_1007.isEmpty()) {
            int next_1007 = s_1007.pop();
            backup_1007.push(next_1007);
            maxValue_1007 = Math.max(maxValue_1007, next_1007);
        }
        while (!backup_1007.isEmpty()) {
            s_1007.push(backup_1007.pop());
        }
        return maxValue_1007;
    }
    public static void main(String[] args) {
        Stack<Integer> s_1007 = new Stack<Integer>();
        s_1007.push(70);
        s_1007.push(12);
        s_1007.push(20);
        System.out.println("Isi stack " + s_1007);
        System.out.println("Stack teratas " + s_1007.peek());
        System.out.println("Nilai Maksimum " + max_1007(s_1007));
    }

}
```

Class NilaiMaksimum_2511531007 bertujuan untuk mencari nilai maksimum dari elemen-elemen yang terdapat dalam sebuah stack. Pada implementasinya, class ini menggunakan stack tambahan sebagai tempat penyimpanan sementara untuk menjaga agar isi stack utama tidak berubah. Proses dimulai dengan mengambil satu elemen sebagai nilai awal maksimum, kemudian seluruh elemen dalam stack dibandingkan satu per satu untuk menemukan nilai terbesar. Setelah proses selesai, seluruh elemen dikembalikan ke stack semula.

2.2.5 Class SiswaStack_2511531007

```java
package pekan3_2511531007;

import java.util.ArrayList;

class Siswa_1007 {
    String nama_1007;
    int nim_1007;

    public Siswa_1007(String nama_1007, int nim_1007) {
        this.nama_1007 = nama_1007;
        this.nim_1007 = nim_1007;
    }
    @Override
    public String toString() {
        return "Nim: " + nim_1007 + ", Nama: " + nama_1007;
    }
}
public class SiswaStack_2511531007 {
    private ArrayList<Siswa_1007> stack_1007;

    public SiswaStack_2511531007() {
        stack_1007 = new ArrayList<>();
    }
    public void push_1007(Siswa_1007 mhs_1007) {
        stack_1007.add(mhs_1007);
    }

    public Siswa_1007 pop_1007() {
        if (!isEmpty_1007()) {
            return stack_1007.remove(stack_1007.size() - 1);
        }
        return null;
    }
```

```java
    public Siswa_1007 peek_1007() {
        if (!isEmpty_1007()) {
            return stack_1007.get(stack_1007.size() - 1);
        }
        return null;
    }

    public boolean isEmpty_1007() {
        return stack_1007.isEmpty();
    }

    public void tampilkanSiswa_1007() {
        for (int i_1007 = stack_1007.size() - 1; i_1007 >= 0; i_1007--) {
            System.out.println(stack_1007.get(i_1007));
        }
    }
    public static void main(String[] args) {
        SiswaStack_2511531007 studentStack_1007 = new SiswaStack_2511531007();

        Siswa_1007 mhs1_1007 = new Siswa_1007("Ali", 1);
        Siswa_1007 mhs2_1007 = new Siswa_1007("Boby", 2);
        Siswa_1007 mhs3_1007 = new Siswa_1007("Charles", 3);

        studentStack_1007.push_1007(mhs1_1007);
        studentStack_1007.push_1007(mhs2_1007);
        studentStack_1007.push_1007(mhs3_1007);

        System.out.println("Siswa di dalam stack: ");
        studentStack_1007.tampilkanSiswa_1007();

        System.out.println("Siswa teratas " + studentStack_1007.peek_1007());
        System.out.println("Mengeluarkan siswa teratas dari stack: " + studentStack_1007.pop_1007());
        System.out.println("Daftar siswa setelah di pop : ");
        studentStack_1007.tampilkanSiswa_1007();
    }

}
```

Class Siswa_1007 merupakan representasi objek yang digunakan untuk menyimpan data siswa, yang terdiri dari atribut nama_1007 dan nim_1007. Class ini memiliki constructor untuk menginisialisasi nilai atribut saat objek dibuat. Selain itu, method toString() dioverride agar data siswa dapat ditampilkan dalam bentuk yang lebih mudah dibaca, yaitu menampilkan NIM dan nama siswa secara bersamaan.

Class SiswaStack_2511531007 merupakan implementasi struktur data stack yang menyimpan objek bertipe Siswa_1007 dengan menggunakan ArrayList sebagai media penyimpanan. Class ini menyediakan method seperti push_1007() untuk menambahkan objek ke dalam stack, pop_1007() untuk menghapus objek teratas, peek_1007() untuk melihat objek teratas tanpa menghapusnya, serta isEmpty_1007() untuk mengecek apakah stack kosong. Selain itu, terdapat method tampilkanSiswa_1007() yang digunakan untuk menampilkan seluruh isi stack dari atas ke bawah.

2.3 Tugas Praktikum

2.3.1 Class Website_2511531007

```java
package tugasPraktikum_pekan3_2511531007;

public class Website_2511531007 {
    String judul_1007;
    String url_1007;

    // Konstruktor
    Website_2511531007 (String judul_1007, String url_1007) {
        this.judul_1007 = judul_1007;
        this.url_1007 = url_1007;
    }

    // Getter
    public String getJudul_1007() {return judul_1007;}
    public String getUrl_1007() {return url_1007;}

    // Setter
    public void setJudul_1007(String judul_1007) {this.judul_1007 = judul_1007;}
    public void setUrl_1007(String url_1007) {this.url_1007 = url_1007;}

    @Override
    public String toString() {
        return "Judul: " + judul_1007 + "\nURL: " + url_1007;
    }

}
```

Class Website_2511531007 merupakan class yang digunakan untuk merepresentasikan data sebuah website. Class ini memiliki dua atribut utama, yaitu judul_1007 yang menyimpan nama atau judul halaman, serta url_1007 yang menyimpan alamat website tersebut. Class ini dilengkapi dengan konstruktor yang berfungsi untuk menginisialisasi nilai judul dan URL saat objek dibuat. Selain itu, terdapat method getter dan setter yang digunakan untuk mengambil dan mengubah nilai atribut secara aman. Method toString() juga dioverride agar objek website dapat ditampilkan dalam format yang lebih mudah dibaca, yaitu menampilkan judul dan URL secara terstruktur. Class ini berperan sebagai tipe data utama yang akan disimpan dalam stack pada program browser.

2.3.2 Class Browser_2511531007

```java
package tugasPraktikum_pekan3_2511531007;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511531007 {
    public static void main(String[] args) {
        // Membuat stack yang menyimpan objek website
        Stack<Website_2511531007> history_1007 = new Stack<>();
        Scanner wb_1007 = new Scanner(System.in);
        int choose_1007;

         do {
             // Menu Tampilan
             System.out.println("\n=== Browser History NIM: 2511531007 ===");
             System.out.println("1. Kunjungi Website (Push)");
             System.out.println("2. Tombol Back (Pop)");
             System.out.println("3. Lihat Halaman Aktif (Peek)");
             System.out.println("4. Keluar");
             System.out.print("Pilihan: ");

             // Input pilihan
             choose_1007 = wb_1007.nextInt();
             wb_1007.nextLine(); //consume newLine

             switch (choose_1007) {
                case 1:
                    // --- KUNJUNGI WEBSITE (PUSH) ---
                    System.out.print("\nMasukkan Judul: ");
                    String judul_1007 = wb_1007.nextLine();
                    System.out.print("Masukkan URL: ");
                    String url_1007 = wb_1007.nextLine();

                    // Buat objek website baru
                    Website_2511531007 newSite_1007 = new Website_2511531007(judul_1007, url_1007);

                    // Masukkan ke stack
                    history_1007.push(newSite_1007);
                    System.out.println("\nBerhasil mengunjungi halaman!");
                    break;
                case 2:
                    // --- TOMBOL BACK (POP) ---
                    if (history_1007.isEmpty()) {
                        System.out.println("History Kosong! Tidak ada halaman sebelumnya.");
                    } else {
                        Website_2511531007 oldSite_1007 = history_1007.pop();
                        System.out.println("Halaman sebelumnya: " + oldSite_1007.getJudul_1007());
                    }
                    break;

                case 3:
                    // --- LIHAT HALAMAN AKTIF (PEEK) ---
                    if (history_1007.isEmpty()) {
                        System.out.println("History Kosong! Tidak ada halaman yang aktif.");
                    } else {
                        Website_2511531007 onSite_1007 = history_1007.peek();
                        System.out.println("\n--- Halaman Saat Ini ---");
                        System.out.println(onSite_1007.toString());
                        System.out.println("------------------------");
                    }
                    System.out.println("Total riwayat: " + history_1007.size() + " halaman.");
                    System.out.println("Status kosong: " + history_1007.isEmpty()); // Cek status history
                    break;

                case 4:
                    System.out.println("Keluar dari browser...");
                    break;

                 default:
                        System.out.println("Pilihan tidak valid!");
             }
         } while (choose_1007 != 4);
         wb_1007.close();
    }

}
```

Class Browser_2511531007 merupakan class utama yang berfungsi untuk menjalankan simulasi sederhana dari browser, khususnya dalam pengelolaan riwayat kunjungan (*history*). Class ini menggunakan struktur data Stack untuk menyimpan objek Website_2511531007, sehingga setiap halaman yang dikunjungi akan disimpan dengan prinsip *Last In First Out* (LIFO). Di dalam method main, program menampilkan menu interaktif yang memungkinkan pengguna untuk melakukan beberapa operasi, seperti mengunjungi website baru (*push*), kembali ke halaman sebelumnya (*pop*), serta melihat halaman yang sedang aktif (*peek*).

Pada saat pengguna memilih untuk mengunjungi website, program akan meminta input judul dan URL, kemudian membuat objek baru dan menambahkannya ke dalam stack. Jika pengguna memilih tombol back, maka elemen teratas dari stack akan dihapus sebagai simulasi kembali ke halaman sebelumnya. Sedangkan fitur peek digunakan untuk menampilkan halaman yang sedang aktif tanpa menghapusnya dari stack. Selain itu, program juga menampilkan jumlah riwayat dan status apakah stack kosong atau tidak. Dengan demikian, class ini menunjukkan penerapan nyata struktur data stack dalam simulasi fitur dasar browser.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan salah satu struktur data yang bekerja dengan prinsip *Last In First Out* (LIFO), di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Stack memiliki operasi dasar seperti *push*, *pop*, dan *peek* yang memungkinkan pengelolaan data secara terstruktur dan efisien.

Implementasi stack dalam praktikum ini dilakukan dengan berbagai pendekatan, seperti menggunakan array, kelas bawaan Java, serta ArrayList. Selain itu, penerapan stack juga ditunjukkan dalam beberapa kasus, seperti pencarian nilai maksimum, evaluasi ekspresi postfix, serta simulasi riwayat browser. Melalui implementasi tersebut, dapat dipahami bahwa stack tidak hanya bersifat teoritis, tetapi juga memiliki banyak aplikasi dalam kehidupan nyata, khususnya dalam pengembangan perangkat lunak.

Dengan demikian, praktikum ini membantu meningkatkan pemahaman mengenai konsep stack serta kemampuan dalam mengimplementasikannya ke dalam program, sehingga dapat digunakan untuk menyelesaikan berbagai permasalahan secara lebih efektif.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Diharapkan asisten praktikum dapat memberikan evaluasi atau umpan balik terhadap laporan maupun program yang telah dibuat, sehingga mahasiswa dapat mengetahui letak kesalahan dan melakukan perbaikan.
4. Dengan adanya latihan yang cukup dan evaluasi yang jelas, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

[1] W. Wahyudi, *Struktur Data dengan Java*. Yogyakarta: CV Eureka Media Aksara, 2025. [Online]. Tersedia: https://books.google.co.id/books?id=EN2lEQAAQBAJ&pg=PA15. [Diakses: 19 April 2026].